

Day07 课堂笔记

复习

递归函数必须要传参吗???

每天时间的都很富裕，老师讲的也很好，如果能慢一点再多举些相关的例子那就最好了，有些测评的题跟课上的内容融不在一起

有些关键字的细节已经忘记了,这个很难搞

老师 就业班还是您来教课吗? 老师 您今年多大了?

```
1  递归函数一般都是有参数.
2  num = 10
3  def func():
4      global
5      if num == 1:
6          return
7      num = num - 1
8      func()
9
10
11 my_list = [20, 30]
12 print(my_list)
13
14 def func(a):
15     a += 20, # 列表, += , extend 即
              a.extend(20,) # str list tuple dict
```

```
16
17 func(my_list)
18 print(my_list) # [20, 30, 20]
19
20 可迭代：能使用 for 循环的就是可迭代的
```

1. 文件读操作

read()

```
1 # 1. 打开文件
2 f = open('a.txt', 'r', encoding='utf-8')
3 # 2. 读写文件 文件对象.read(n) n 一次读取多少字节的内容, 默认不写, 读取全部内容
4 buf = f.read(3) 123
5 print(buf) # 123 -----
6 print('-'*30) 4
7 buf = f.read(3) # 5
8 print(buf)
9 # 3. 关闭文件
10 f.close()
11
```

按行读取

```
1 f = open('a.txt', 'r', encoding='utf-8')
2 # f.readline() # 一次读取一行的内容, 返回值是读取到的内容(str)
3 # buf = f.readline()
4
5 # f.readlines() # 按行读取, 一次读取所有行, 返回值是列表, 列表中的每一项是一个字符串, 即一行的内容
6 buf = f.readlines()
7 print(buf)
8 f.close()
```

Run: 02-按行读取文件 ×

C:\Develop\python38\python.exe C:/Users/n1/Desktop/43/day07/02-按行读取文件.py
['1234\n', '5678']

模拟读取大文件

- 1 read() 一次读取全部的内容
- 2 read() 读到文件末尾会返回空

```
1 f = open('b.txt', 'r', encoding='utf-8')
2 while True:
3     buf = f.read(5) # f.read(4096)
4     if buf:
5         print(buf, end='')
6     else:
7         break
8
9 f.close()
10
```

```
1 f = open('a.txt', 'r', encoding='utf-8')
2 while True:
3     buf = f.readline()
4     if buf: # if len(buf) > 0 容器, 可以直接作
5         print(buf, end='')
6     else:
7         # 文件读完了
8         break
9
10 f.close()
11
```

补充: 计算机中的大小

- 1 计算机只认识 0 和 1 ,
- 2 进制:
- 3 二进制, 只有 0 和 1 组成, 逢二进一

4 十进制, 0 1 2 3 4 5 6 7 8 9 , 逢十进一
5 十六进制, 0 1 2 3 4 5 6 7 8 9 A B C D E F
 逢十六进一
6 计算机最小的单位是 bit(位), 只有两个数据值 0 和 1
7 字节: 是计算机中基本的储存单位 byte
8 1 byte = 8 bit
9 1KB = 1024 byte
10 1MB = 1024 KB
11 1GB = 1024 MB
12 1TB = 1024 GB
13 ...
14
15 100Mbits 200Mbits
16 12.5MB/s 25MB/s

文件打开模式

- 1 文本文件：txt, .py .md 能够使用记事本打开的文件
- 2 二进制文件：具有特殊格式的文件，mp3 mp4 rmvb avi
png jpg 等
- 3
- 4 文本文件可以使用 文本方式打开文件，也可以使用二进制的方式打开文件
- 5
- 6 二进制文件，只能使用二进制的方式打开文件
- 7 二进制打开方式如下：不管读取，还是书写，都需要使用二进制的
数据
- 8 rb wb ab
- 9 注意点：不能指定 encoding 参数

```
1 f = open('c.txt', 'wb')
2 f.write('你好'.encode()) # encode() 将str 转换为二进制格式的字符串
3 f.close()
4
5
6 f1 = open('c.txt', 'rb')
7 buf = f1.read()
8 print(buf)
9 print(buf.decode())
10 f1.close()
```

访问模式	说明
r	以只读方式打开文件。文件的指针将会放在文件的开头。这是默认模式。

w	打开一个文件只用于写入。如果该文件已存在则将其覆盖。如果该文件不存在，创建新文件。
a	打开一个文件用于追加。如果该文件已存在，文件指针将会放在文件的结尾。也就是说，新的内容将会被写入到已有内容之后。如果该文件不存在，创建新文件进行写入。
rb	以二进制格式打开一个文件用于只读。文件指针将会放在文件的开头。这是默认模式。
wb	以二进制格式打开一个文件只用于写入。如果该文件已存在则将其覆盖。如果该文件不存在，创建新文件。
ab	以二进制格式打开一个文件用于追加。如果该文件已存在，文件指针将会放在文件的结尾。也就是说，新的内容将会被写入到已有内容之后。如果该文件不存在，创建新文件进行写入。
r+	打开一个文件用于读写。文件指针将会放在文件的开头。
w+	打开一个文件用于读写。如果该文件已存在则将其覆盖。如果该文件不存在，创建新文件。

a+	打开一个文件用于读写。如果该文件已存在，文件指针将会放在文件的结尾。文件打开时会是追加模式。如果该文件不存在，创建新文件用于读写。
rb+	以二进制格式打开一个文件用于读写。文件指针将会放在文件的开头。
wb+	以二进制格式打开一个文件用于读写。如果该文件已存在则将其覆盖。如果该文件不存在，创建新文件。
ab+	以二进制格式打开一个文件用于追加。如果该文件已存在，文件指针将会放在文件的结尾。如果该文件不存在，创建新文件用于读写。

应用-文件备份

- 1 1. 用只读的方式, 打开文件
- 2 2. 读取文件内容
- 3 3. 关闭文件
- 4 4. 只写的方式, 打开新文件
- 5 5. 将 第 2 步读取的内容写入新文件
- 6 6. 关闭新文件
- 7
- 8 思考:
- 9 1. 如果文件比较大, 循环读取文件
- 10 2. 复制备份的文件可能是 txt 文件, 可能是 二进制文件, ---> 使用二进制方式打开文件

```
>>> file_name = 'a.txt'
>>> file_name.rfind('.')
1
>>> index = file_name.rfind('.')
>>> file_name[index: ]
'.txt'
>>> file_name[:index]
'a'
```

```

1  file_name = input('请输入要备份的文件名')
2
3  # 1. 用只读的方式, 打开文件
4  f = open(file_name, 'rb')
5  # 2. 读取文件内容
6  buf = f.read()
7  # 3. 关闭文件
8  f.close()
9
10 # 根据原文件名, 找到文件后缀和文件名
11 index = file_name.rfind('.')
12 # 后缀 file_name[index:]
13 # 新文件名 文件名 后缀
14 new_file_name = file_name[:index] + '[备份]' + file_name[index:]
15 print(new_file_name)
16 # 4. 只写的方式, 打开新文件
17 f_w = open(new_file_name, 'wb')
18 # 5. 将 第 2 步读取的内容写入新文件
19 f_w.write(buf)
20 # 6. 关闭新文件
21 f_w.close()
22
23

```

文件和文件夹的操作

```

1  # 对文件和目录的操作, 需要导入 os 模块
2  import os
3
4  # 1. 文件重命名 os.rename(原文件路径名, 新文件路径名)
5  # os.rename('a.txt', 'aa.txt')
6  # 2. 删除文件 os.remove(文件的路径名)
7  # os.remove('aa.txt')
8  # 3. 创建目录 os.mkdir(目录路径名) make directory
9  # os.mkdir('test')
10 # os.mkdir('test/aa')
11
12 # 4. 删除空目录 os.rmdir(目录名) remove directory
13 # os.rmdir('test/aa')
14 # 5. 获取当前所在的目录 os.getcwd() get current working directory
15 buf = os.getcwd()
16 print(buf)
17 # 6. 修改当前的目录 os.chdir(目录名) change dir
18 os.chdir('test')
19 buf = os.getcwd()
20 print(buf)
21 # 7. 获取指定目录中的内容, os.listdir(目录), 默认不写参数, 是获取当前目录中的内容
22 # 返回值是列表, 列表中的每一项是文件名
23 buf = os.listdir() # test
24 print(buf)

```

应用-批量修改文件名

```
1  import os
2
3
4  def create_files():
5      for i in range(10):
6          file_name = 'test/file_' + str(i) + '.txt'
7          print(file_name)
8          f = open(file_name, 'w')
9          f.close()
10
11
12  def create_files_1():
13      os.chdir('test')
14      for i in range(10, 20):
15          file_name = 'file_' + str(i) + '.txt'
16          print(file_name)
17          f = open(file_name, 'w')
18          f.close()
19  os.chdir('../') # ../ 上一级目录
```

```
22 def modify_filename():
23     os.chdir('test')
24     buf_list = os.listdir()
25     # print(buf_list)
26     for file in buf_list:
27         new_file = 'py43_' + file
28         os.rename(file, new_file)
29
30     os.chdir('../')
31
32
33 def modify_filename_1():
34     os.chdir('test')
35     buf_list = os.listdir()
36     # print(buf_list)
37     for file in buf_list:
38         num = len('py43_')
39         new_file = file[num:]
40         os.rename(file, new_file)
41
42     os.chdir('../')
43
44
```

学生管理系统文件版

```
1  学生管理系统
2  {'name': 'isaac', 'age': 18, 'gender': m}
3
4  [{}, {}]
5  需要将所有的学生信息保存的到文件中.最简单的方法,是
   将整个列表保存到文件中
6  '[{}, {}, {}]'
7  w
8
9  '[{}, {}, {}]'    ---> [{}, {}, {}]
10 eval()
```

```
86  def save():
87      # 1. 打开文件
88      f = open('student.txt', 'w')
89      f.write(str(stu_list))
90      f.close()
91
```

```
1  '[{}, {}, {}]'    ---> eval('[{}, {}, {}]')
2  stu_list
```

```
95 def load_file():
96     global stu_list
97     if os.path.exists('student.txt'):
98         f = open('student.txt', 'r', encoding='utf-8')
99         buf = f.read()
100         if buf:
101             stu_list = eval(buf)
102         f.close()
103
104
105 def main():
106     load_file() # 只执行一次
107     while True:
```

面向对象

概念介绍[了解]

面向对象和面向过程,是两种编程思想. 编程思想是指对待同一个问题,解决问题的套路方式.

- 面向过程: 注重的过程,实现的细节.亲力亲为.
 - 函数
- 面向对象: 关注的是结果, 偷懒.
 - 封装, 继承 和多态(三大技术)

类和对象[理解]

类和对象,是面向对象中非常重要的两个概念

- 类
 - 类是泛指的,指代多个,是由对的抽象来的
 - 在代码中,是通过关键字 `class` 定义的
- 对象
 - 对象是特指的,指代一个,是具体存在的
 - 在代码中,对象是由类创建的

类的组成

- 类名
- 属性: 特性, 变量
 - 一个类的属性有很多,值关注需要使用的,没有用到的不用管
- 方法: 行为, 函数

类的抽象

- 名词提炼法
- 所见即所得

类的定义

```
1  随后的代码,使用狗类进行演示说明
2  类名: Dog(狗类)
3  属性:
4      姓名 name
5      年龄 age
6      ....
7  方法:
8      吃 eat()
9      玩 play()
10     ....
```

```
8  [ ]
9  [ ]
10 # 定义方式一, 推荐
11 class Dog(object):
12     [ ]
13     [ ]
14     [ ]
15 # 定义方式二
16 class Dog1(): # 括号中的内容可以不写
17     [ ]
18     [ ]
19     [ ]
20 # 定义方式三
21 class Dog2: # 括号也可以不写
22     [ ]
23     [ ]
24     [ ]
25 """
26 新式类: 直接或者间接继承object的类, 在python3中,所有的类默认继承object类,即python3中所有的类都是新式类
27 旧式类(经典类): 已经过时,不推荐使用
28 """
```

PEP 8 代码规范: 类定义的前后, 需要两个空行

```
1  # 在python中, 定义类使用关键字class, 语法如下
2  """
3  # object 是所有的类基类, 即最初始的类
4  # 类名: 遵循大驼峰的命名规范
5  class 类名(object):
6      类中的代码
7  """
```

创建对象

在代码中,对象是由类创建的.

```
1  # 定义类
2  class Dog(object):
3      # 在类中定义的函数,称为方法, 函数的所有知识都可以使用
4      def play(self): # 暂时不管self ,
5          print('小狗快乐的拆家中.....')
6
7
8  # 创建对象 变量 = 类名()
9  dog = Dog() # 创建一个对象,dog
10 print(id(dog))
11
12 dog1 = Dog() # 创建一个对象 dog1
13 print(id(dog1))
14
15 # 可以使用对象调用类中的方法. 对象.方法名()
16 dog.play()
17 dog1.play()
18
```